

MARWADI UNIVERSITY

DEPARTMENT OF AI, ML

DAA (01AI0506) - Lab Practical

Practical 01: Implementation and Time Analysis of Sorting Algorithms

Name: K.Raju Enrollment No: 92460118432

Aim

Implementation and Time Analysis of Sorting Algorithms as specified in the DAA Lab Manual.

Program

```
#include <iostream>
#include <vector>
using namespace std;

void bubbleSort(int a[], int n) {
    for (int i = 0; i < n - 1; i++) {
        bool swapped = false;
        for (int j = 0; j < n - i - 1; j++) {
            if (a[j] > a[j + 1]) {
                swap(a[j], a[j + 1]);
                swapped = true;
            }
        }
        if (!swapped) break;
    }
}

void selectionSort(int a[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int minIndex = i;
        for (int j = i + 1; j < n; j++)
            if (a[j] < a[minIndex]) minIndex = j;
        swap(a[i], a[minIndex]);
    }
}

void insertionSort(int a[], int n) {
    for (int i = 1; i < n; i++) {
        int key = a[i], j = i - 1;
        while (j >= 0 && a[j] > key) {
            a[j + 1] = a[j];
            j--;
        }
        a[j + 1] = key;
    }
}

void merge(int a[], int l, int m, int r) {
    vector<int> temp;
    int i = l, j = m + 1;
    while (i <= m && j <= r)
        temp.push_back(a[i] <= a[j] ? a[i++] : a[j++]);
    while (i <= m) temp.push_back(a[i++]);
    while (j <= r) temp.push_back(a[j++]);
    for (int k = 0; k < (int)temp.size(); k++) a[l + k] = temp[k];
}

void mergeSort(int a[], int l, int r) {
    if (l >= r) return;
    int m = l + (r - l) / 2;
    mergeSort(a, l, m);
    mergeSort(a, m + 1, r);
    merge(a, l, m, r);
}

int partitionArray(int a[], int low, int high) {
```

```

        int pivot = a[high], i = low - 1;
        for (int j = low; j < high; j++) {
            if (a[j] <= pivot) {
                i++;
                swap(a[i], a[j]);
            }
        }
        swap(a[i + 1], a[high]);
        return i + 1;
    }

    void quickSort(int a[], int low, int high) {
        if (low < high) {
            int p = partitionArray(a, low, high);
            quickSort(a, low, p - 1);
            quickSort(a, p + 1, high);
        }
    }

    void printArray(int a[], int n) {
        for (int i = 0; i < n; i++) cout << a[i] << " ";
        cout << endl;
    }

    int main() {
        int original[] = {64, 34, 25, 12, 22, 11, 90, 45, 78, 3};
        int n = sizeof(original) / sizeof(original[0]);

        int a[10];

        copy(original, original + n, a);
        bubbleSort(a, n);
        cout << "Bubble Sort: "; printArray(a, n);

        copy(original, original + n, a);
        selectionSort(a, n);
        cout << "Selection Sort: "; printArray(a, n);

        copy(original, original + n, a);
        insertionSort(a, n);
        cout << "Insertion Sort: "; printArray(a, n);

        copy(original, original + n, a);
        mergeSort(a, 0, n - 1);
        cout << "Merge Sort: "; printArray(a, n);

        copy(original, original + n, a);
        quickSort(a, 0, n - 1);
        cout << "Quick Sort: "; printArray(a, n);
    }
}

```

Output

```

Bubble Sort: 3 11 12 22 25 34 45 64 78 90
Selection Sort: 3 11 12 22 25 34 45 64 78 90
Insertion Sort: 3 11 12 22 25 34 45 64 78 90
Merge Sort: 3 11 12 22 25 34 45 64 78 90
Quick Sort: 3 11 12 22 25 34 45 64 78 90

```

Time and Space Analysis

Bubble: Best $\Theta(n)$, Average $\Theta(n^2)$, Worst $\Theta(n^2)$, Space $\Theta(1)$. Selection: Best/Average/Worst $\Theta(n^2)$, Space $\Theta(1)$. Insertion: Best $\Theta(n)$, Average/Worst $\Theta(n^2)$, Space $\Theta(1)$. Merge: Best/Average/Worst $\Theta(n \log n)$, Space $\Theta(n)$. Quick: Best/Average $\Theta(n \log n)$, Worst $\Theta(n^2)$; recursion stack Average $\Theta(\log n)$, Worst $\Theta(n)$.

Result

The practical was implemented successfully and the expected output was obtained.